

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250130>

Outline : Support Vector Machine

- Training models (more?)
- Support Vector Machine
- Linear SVM Classification
 - Soft Margin Classification
- Nonlinear SVM Classification
 - Polynomial Kernel
 - Similarity Features
 - Gaussian RBF Kernel
 - Computational Complexity
- SVM Regression
- Examples

—

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019 ([Available online](#)) / **Chapter 5**

Support Vector Machine (SVM)

- Can be used for linear or nonlinear classification, regression, and even outlier detection
- Works well on complex but small- or medium-sized datasets.

Iris virginica type based on petal width and length



Figure 4-22. Flowers of three iris plant species¹⁶

Linear SVM Classification (1) - Steps 1-4

SVM classifier fitting “the widest possible street” (represented by the parallel dashed lines) between the classes.

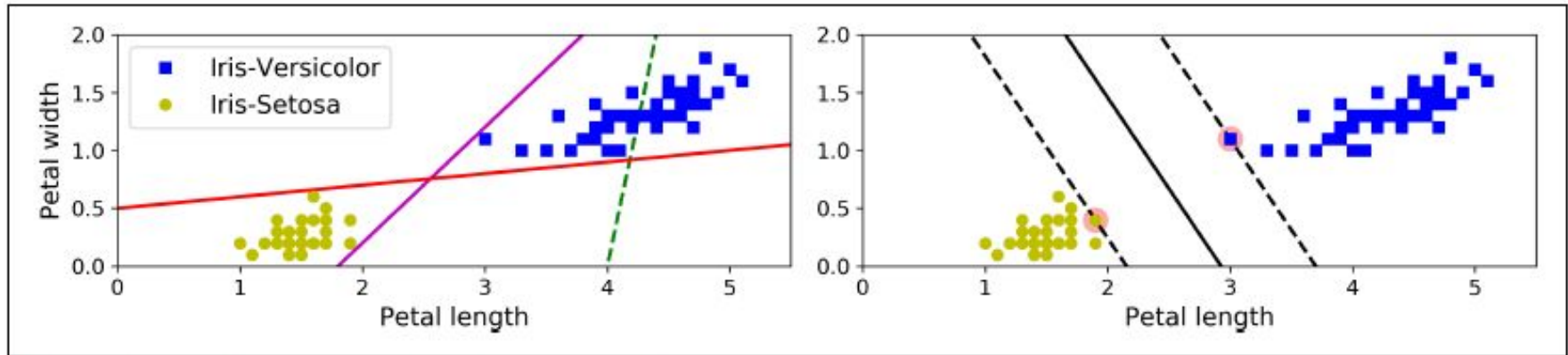


Figure 5-1. Large margin classification

Linear SVM Classification (2)

Adding more training instances “off the street” will not affect the decision boundary: it is fully determined (“supported”) by the instances located on the edge of the “street”.

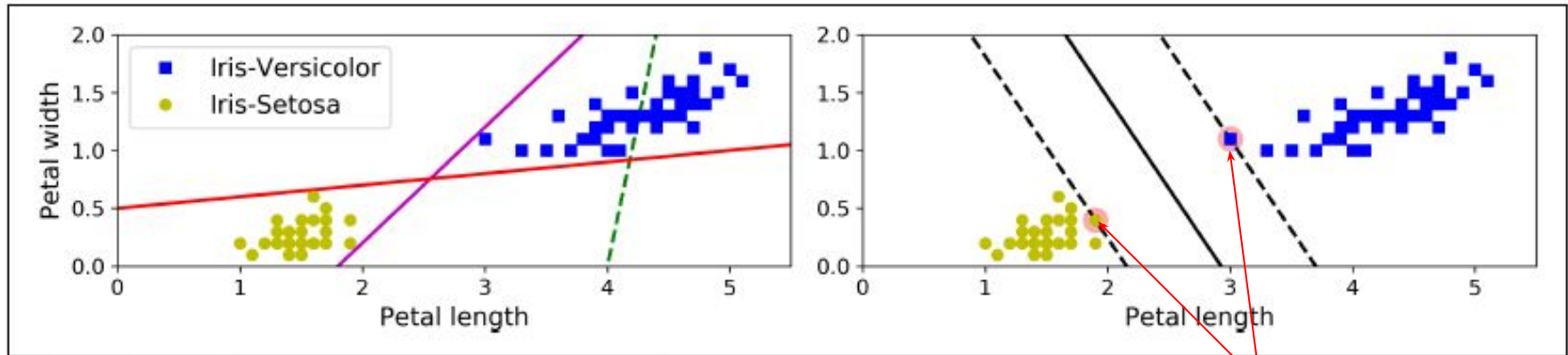


Figure 5-1. Large margin classification

'support vectors'

Sensitivity to feature scales

SVMs are sensitive to the feature scales. Use [StandardScaler](#).

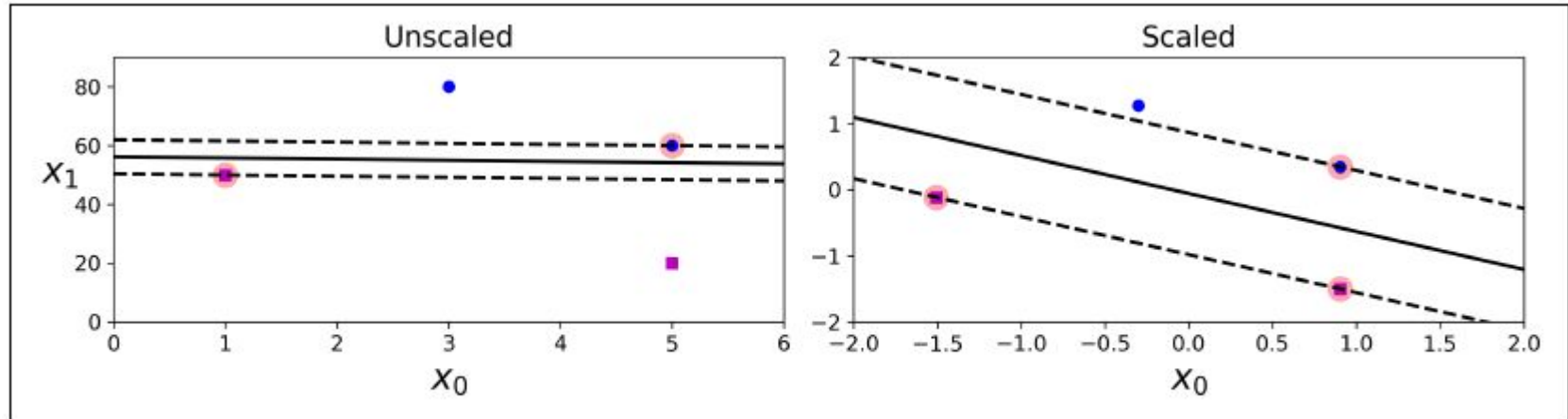
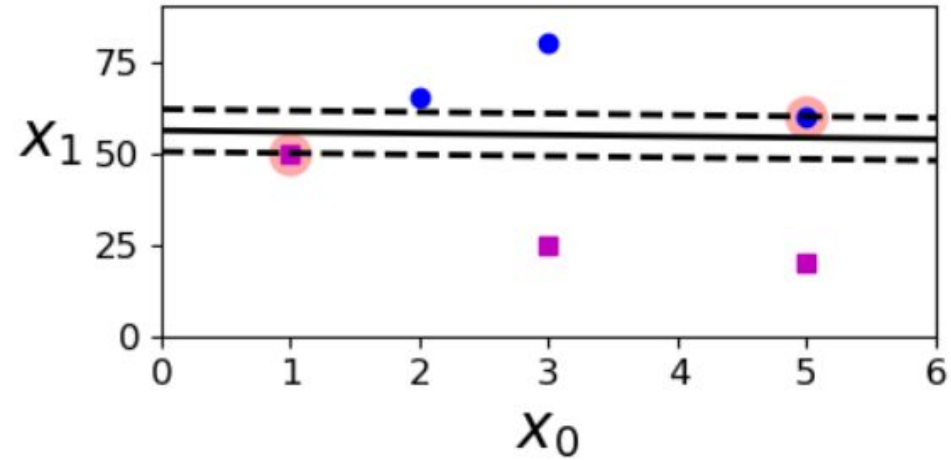
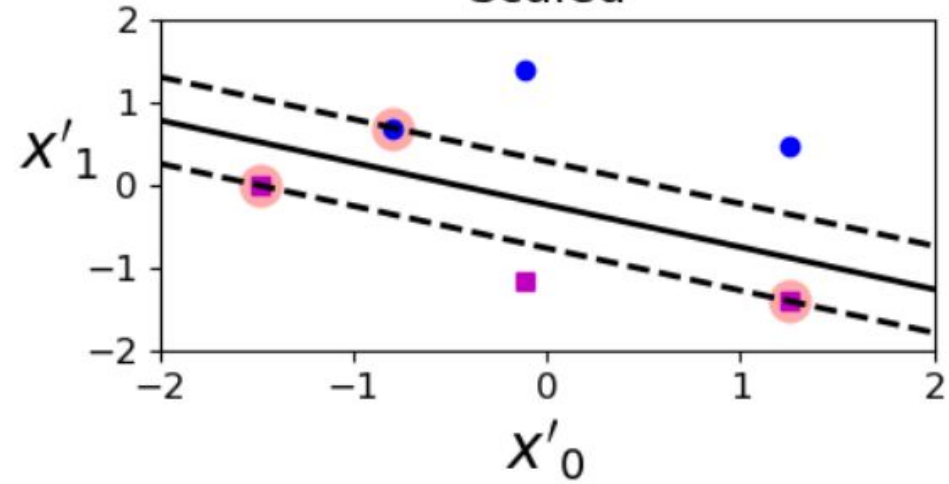


Figure 5-2. Sensitivity to feature scales

Unscaled



Scaled



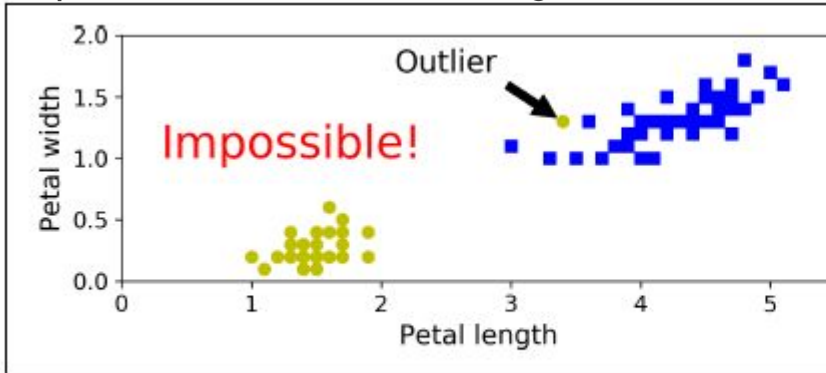
Soft Margin Classification

Steps 5-10

Soft Margin Classification (1)

- Hard margin classification: All instances must be off the street and on the right side (correctly classified).
 - works only if data is linearly separable;
 - sensitive to outliers.

impossible to find a hard margin



will not generalize?:

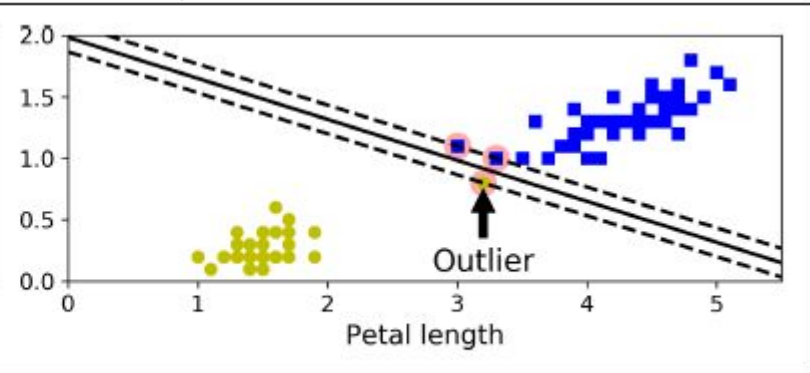


Figure 5-3. Hard margin sensitivity to outliers

Soft Margin Classification (2)

A more flexible model? Good balance between keeping the street as large as possible and limiting the margin violations (i.e., instances that end up in the middle of the street or even on the wrong side) → *Soft margin classification*.

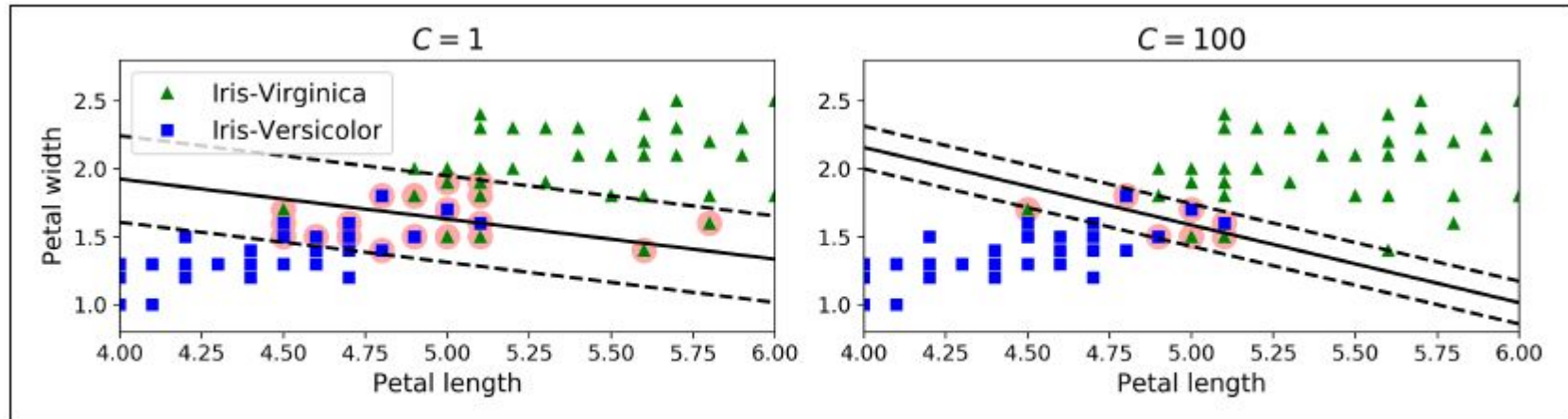


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Soft Margin Classification (3) - SVC

C hyperparameter: **smaller C** - wider street, but more violations; **larger C** - fewer margin violations, but a smaller margin.

If the SVM model is overfitting, try regularizing it by reducing C .

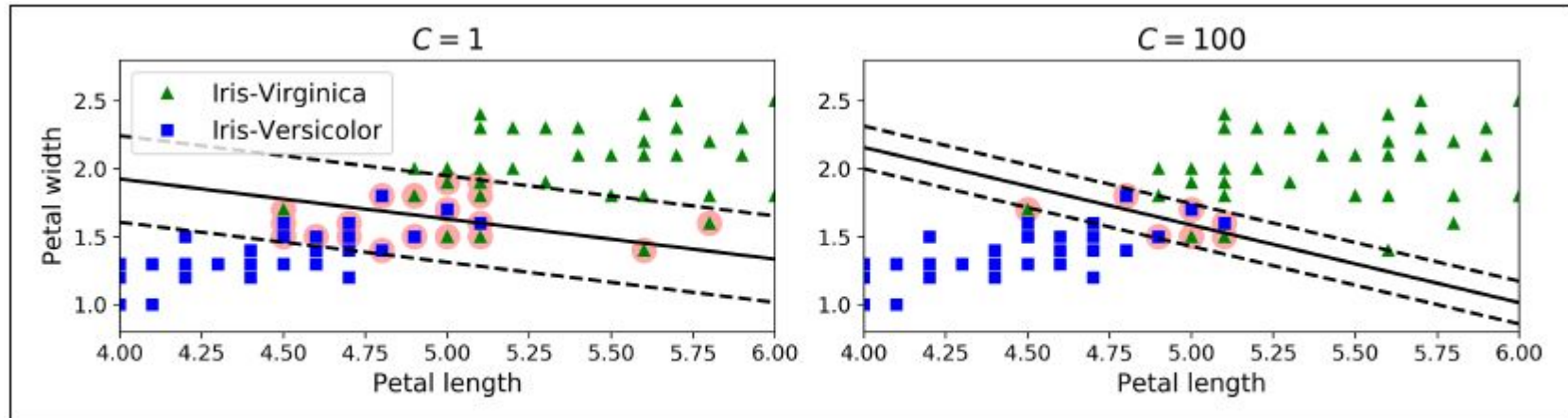


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Soft Margin Classification (4)

- LinearSVC : more flexibility in the choice of penalties and loss functions and should scale better to large numbers of samples.
- Alternatively, `SVC(kernel="linear", C=1)` : slower.
- SGDClassifier : `SGDClassifier(loss="hinge", alpha=1/(m*C))` - regular Stochastic Gradient Descent (L 4) to train a linear SVM classifier. Is not as fast as LinearSVC, but can be useful for huge data sets (out-of-core training)-

Nonlinear SVM Classification

Steps 11-14

Nonlinear SVM Classification (1)

Not only linearly separable datasets. So, adding polynomial features.

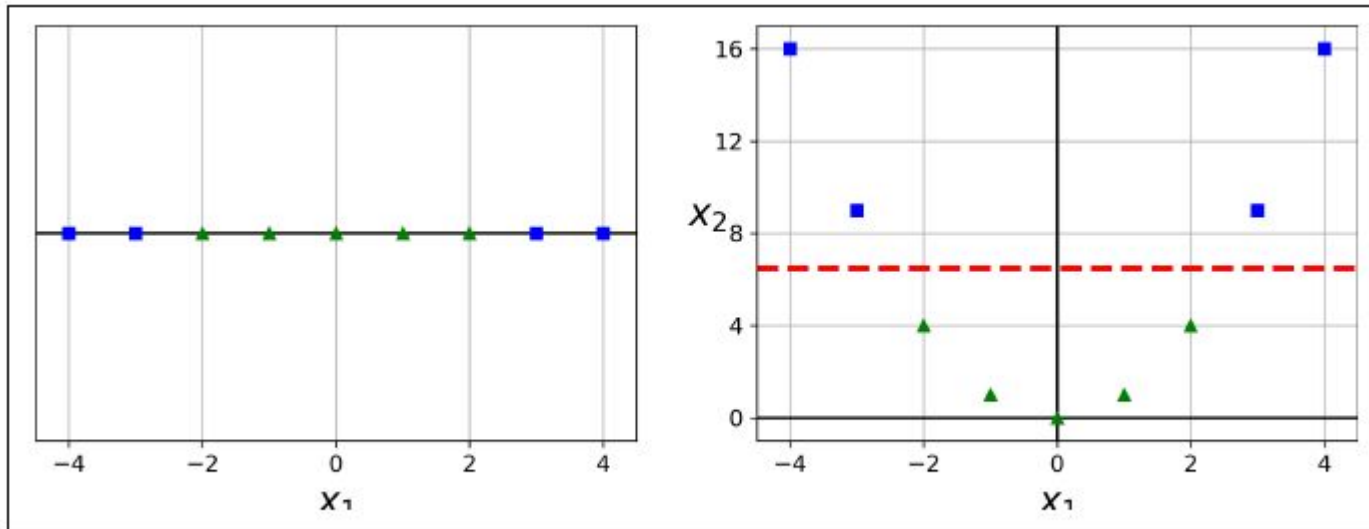


Figure 5-5. Adding features to make a dataset linearly separable

Nonlinear SVM Classification (2)

Pipeline

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=3)),
    ("scaler", StandardScaler()),
    ("svm_clf", LinearSVC(C=10, loss="hinge"))
])

polynomial_svm_clf.fit(X, y)
```

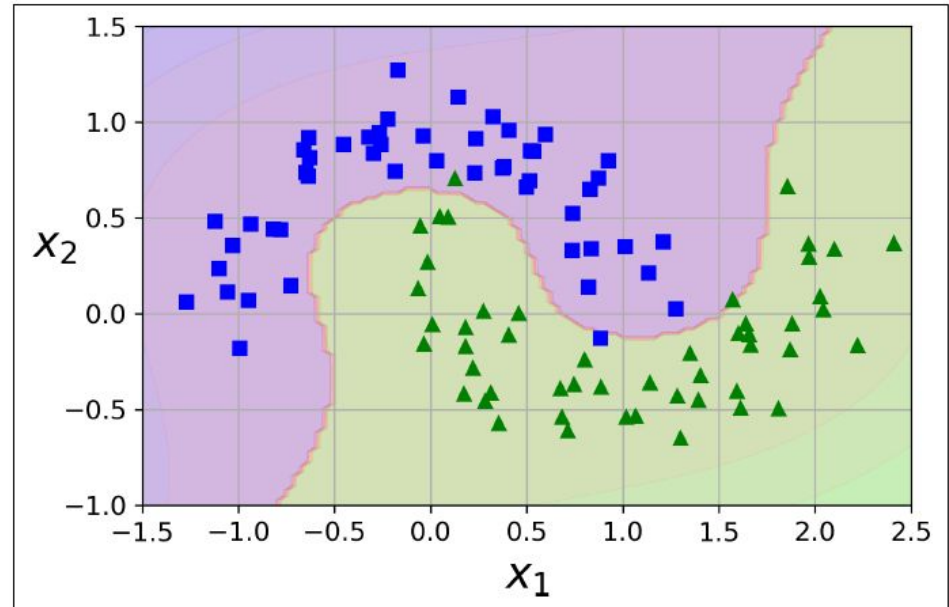


Figure 5-6. Linear SVM classifier using polynomial features

Polynomial Kernel

Steps 15-17

Kernel trick

- a low polynomial degree : cannot deal with very complex datasets
- a high polynomial degree : creates a huge number of features, making the model too slow.

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```

Polynomial Kernel

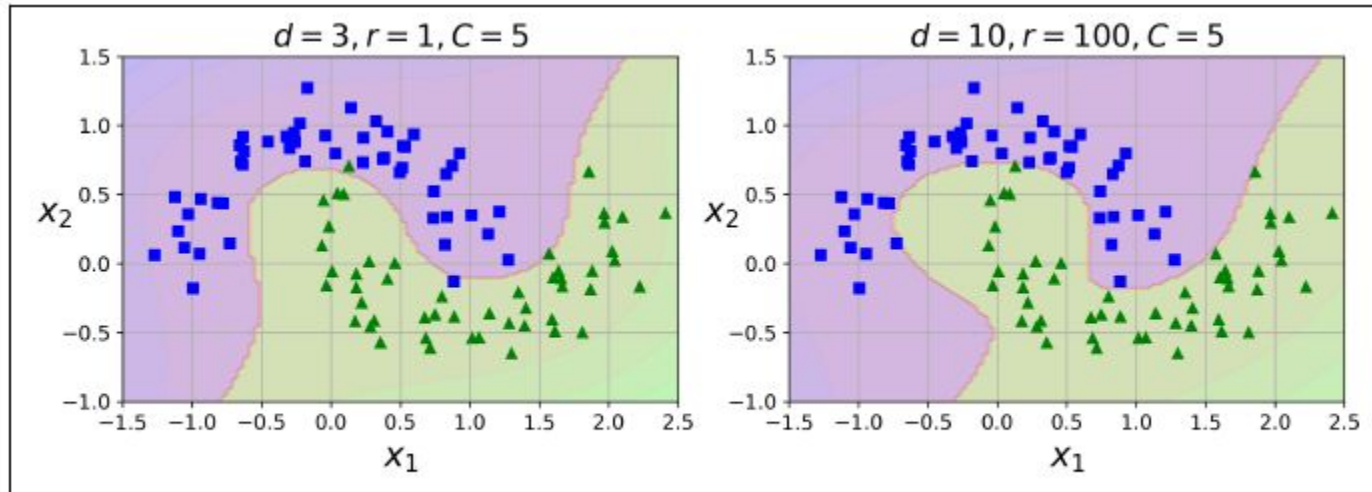


Figure 5-7. SVM classifiers with a polynomial kernel

Similarity Features

Steps 18-19

Similarity Features (1)

Add features computed using a *similarity function*. Measures how much each instance resembles a particular *landmark*. Def.: [Exp](#).

Example: two landmarks to it at $x_1 = -2$ and $x_1 = 1$

Similarity function: Gaussian Radial Basis Function (RBF) with γ (gamma) = 0.3

Equation 5-1. Gaussian RBF

$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp \left(-\gamma \| \mathbf{x} - \ell \|^2 \right)$$

Similarity Features (2)

For $x_1 = -1$:
 $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$
 $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$.

Equation 5-1. Gaussian RBF

$$\phi_\gamma(\mathbf{x}, \ell) = \exp(-\gamma \|\mathbf{x} - \ell\|^2)$$

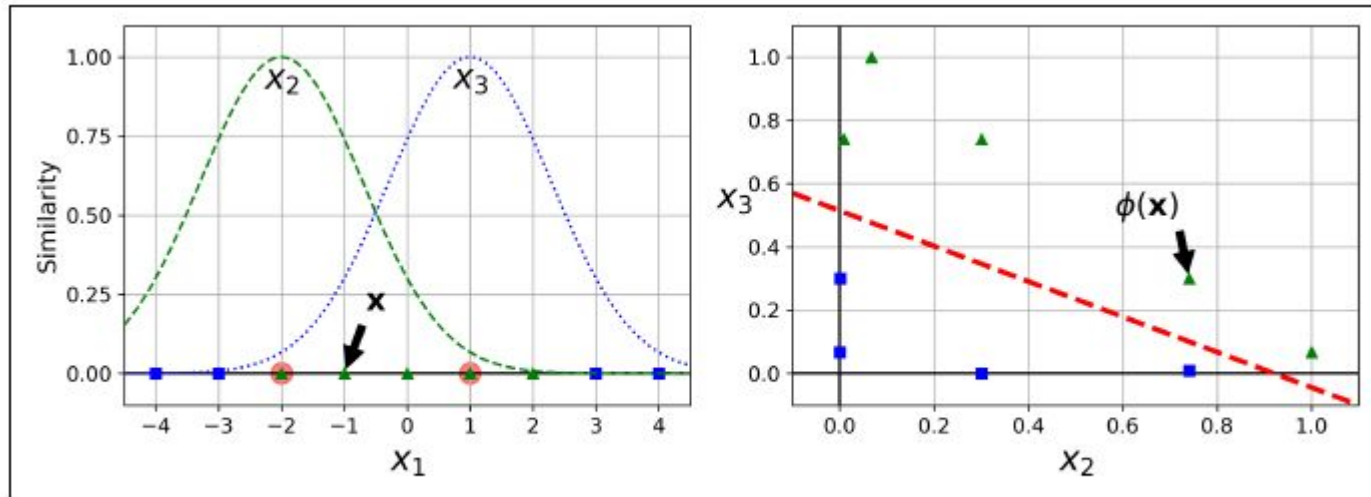


Figure 5-8. Similarity features using the Gaussian RBF

Similarity Features (3)

Equation 5-1. Gaussian RBF

$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp \left(-\gamma \| \mathbf{x} - \ell \|^2 \right)$$

How to select landmarks?

The simplest approach is to create a landmark at the location of each and every instance in the dataset. This creates many dimensions and thus increases the chances that the transformed training set will be linearly separable.

The downside is that a training set with m instances and n features gets transformed into a training set with m instances and m features (assuming you drop the original features). If your training set is very large, you end up with an equally large number of features.

Gaussian RBF Kernel

Steps 20-21

Gaussian RBF Kernel (1)

... Still it may be computationally expensive to compute all the additional features, especially on large training sets. So, Gaussian RBF using the SVC class:

```
rbf_kernel_svm_clf = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))  
])  
rbf_kernel_svm_clf.fit(X, y)
```

Gaussian RBF Kernel (2)

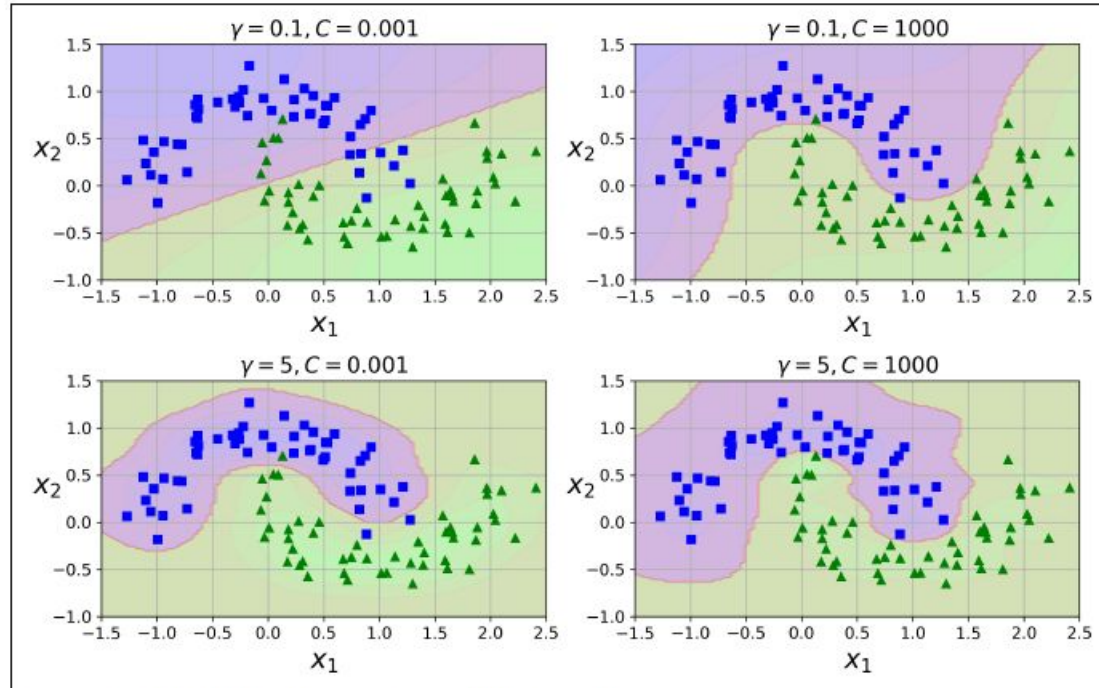


Figure 5-9. SVM classifiers using an RBF kernel

How to decide which kernel to use?

- Always try the **linear kernel first** (LinearSVC is much faster than SVC(kernel="linear")), especially if the training set is very large or if it has plenty of features.
- If the training set is not too large, try the **Gaussian RBF** kernel as well; it works well in most cases.
- Then if there is spare time and computing power, you can also experiment with a few other kernels using cross-validation and grid search, especially if there are kernels specialized for your training set's data structure.
 - String kernels
 - Levenshtein distance

Computational complexity

Table 5-1. Comparison of Scikit-Learn classes for SVM classification

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SGDClassifier	$O(m \times n)$	Yes	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

SVM Regression

Steps 22-29

SVM Regression (1)

In addition to linear and nonlinear classification, also linear and nonlinear **regression**.

Trying to fit instances on the street between two classes while limiting margin violations.

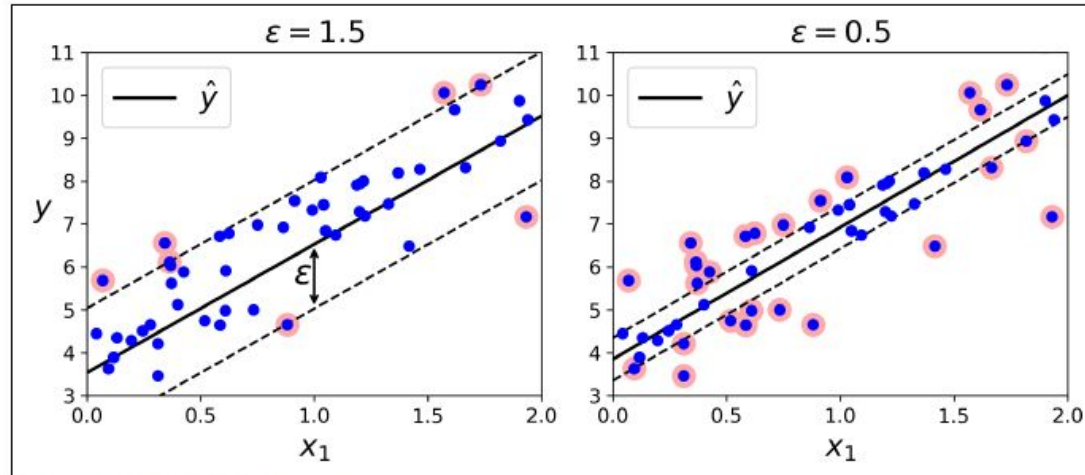


Figure 5-10. SVM Regression

SVM Regression (2)

```
eps_x1 = 0  
eps_y_pred = svm_reg1.predict([[eps_x1]])  
poly1_y_pred = svm_poly_reg1.predict([[eps_x1]])  
poly2_y_pred = svm_poly_reg2.predict([[eps_x1]])  
eps_y_pred, poly1_y_pred, poly2_y_pred  
  
(array([3.53362371]), array([0.16764293]), array([0.22242492]))
```

<:Making predictions

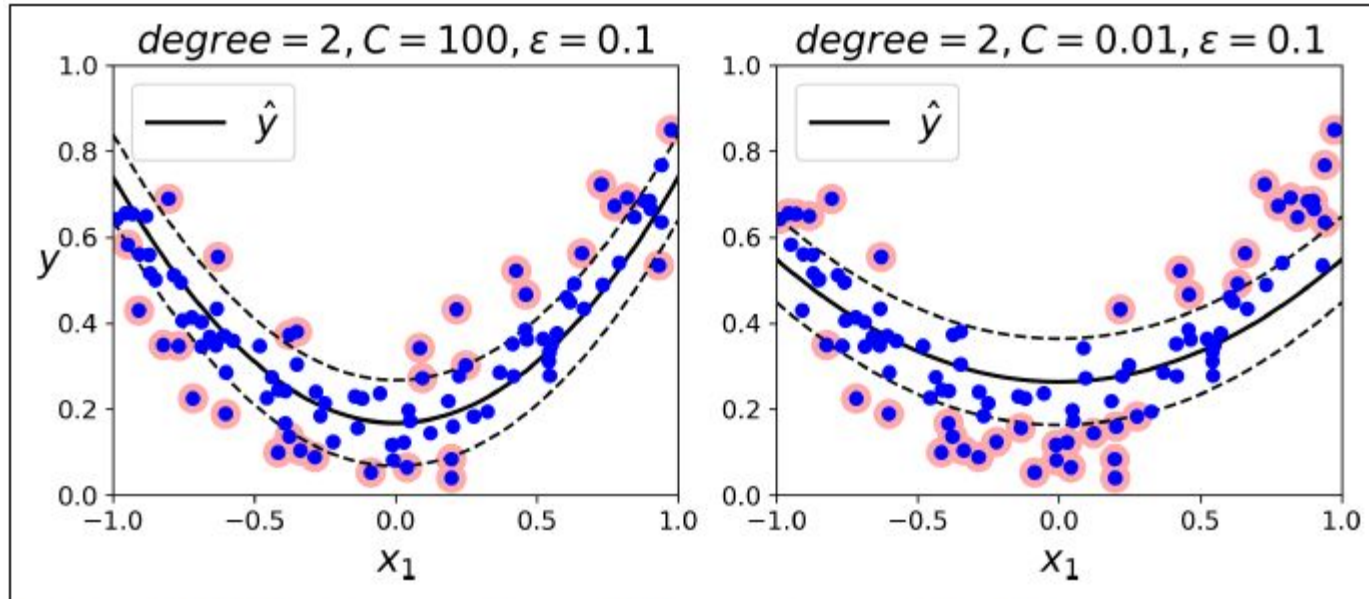
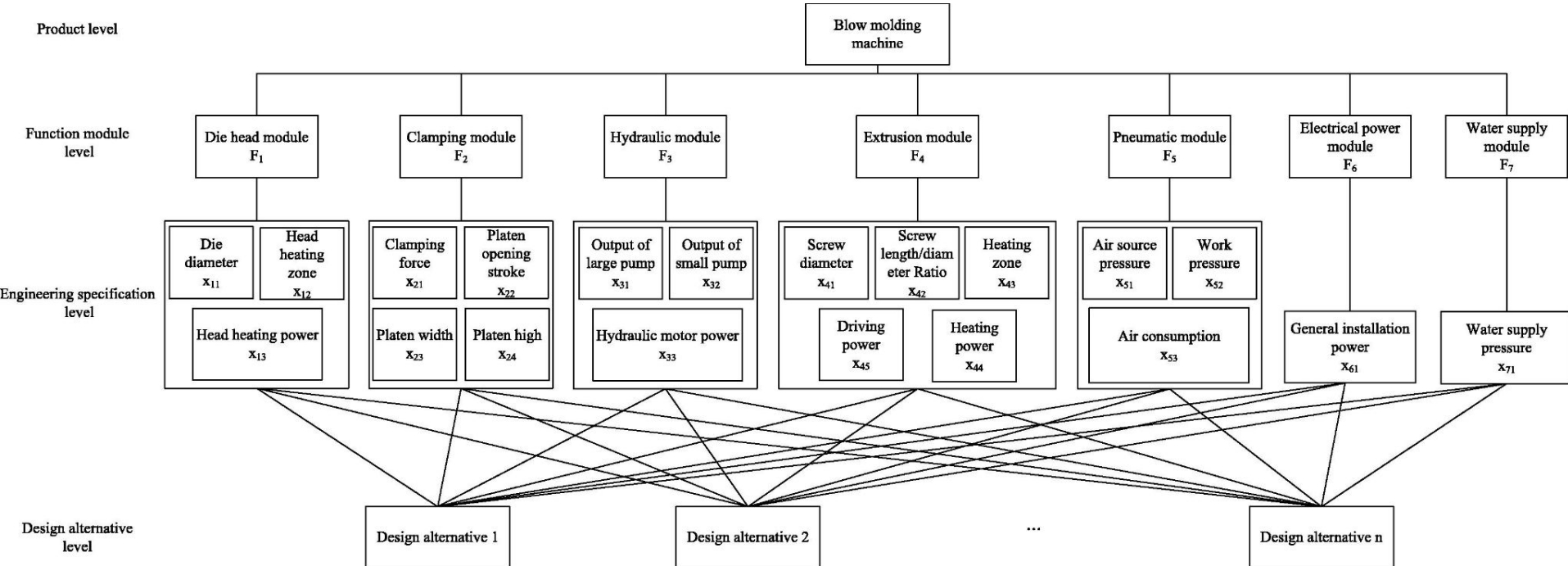


Figure 5-11. SVM regression using a 2nd-degree polynomial kernel

A quantitative approach to design
alternative evaluation based on data-driven
performance prediction

Problem: Design of Blow molding machine



The framework

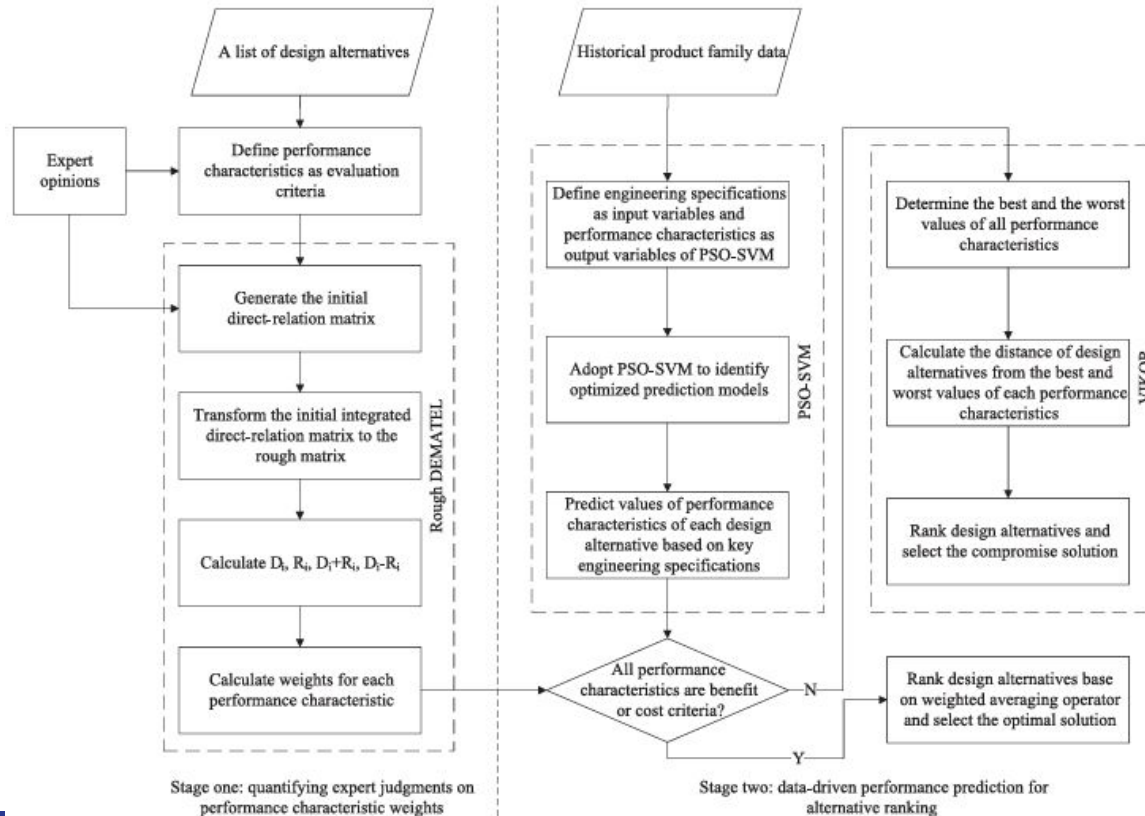


Fig. 1. Framework of the proposed method.

Takeaways

- SVM is a versatile supervised learning algorithm that can be used for classification, regression and anomaly / outlier detection.
- Different ways to add features with a similarity function.
- Works well on small- or medium-sized datasets. With SGDClassifier can also support out-of-core training.